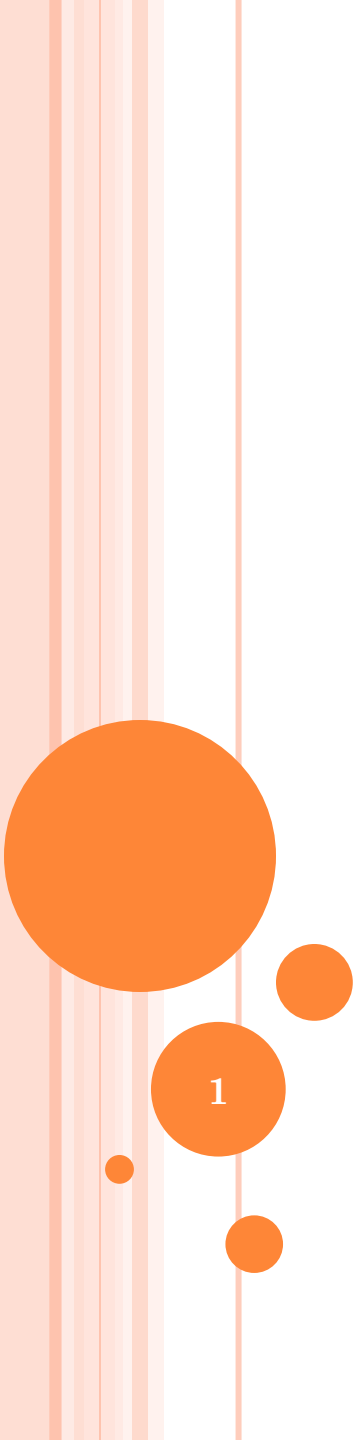




SISTEME DE OPERARE

Curs 6: Thread-uri (Fire de execuție)

Kinga Márton

Universitatea Tehnică din Cluj-Napoca
Departamentul de Calculatoare

○ În cuprins

- Rezumat procese
- Aspecte generale legate de gestiunea threadurilor
- Exemple Pthreads pentru gestiunea threadurilor în Linux

PROCESE

PROCESUL

- Program în execuție
- Program vs. proces
- **Scop:** rularea mai multor joburi pe sistem
 - În mod aparent simultan
 - Fără a ține cont de disponibilitatea procesorului
- **Provocare:** crearea iluziei că există o multitudine de procesoare (chiar dacă există unul singur sau un număr redus)
- **Soluție:** virtualizarea procesorului
 - Time sharing (multiplexare în timp)
 - Se rulează un proces – se oprește – se rulează alt proces – se reia primul proces, șamd.
 - Permite utilizatorilor rularea unui număr foarte mare de procese aparent simultan
 - Cost: performanță

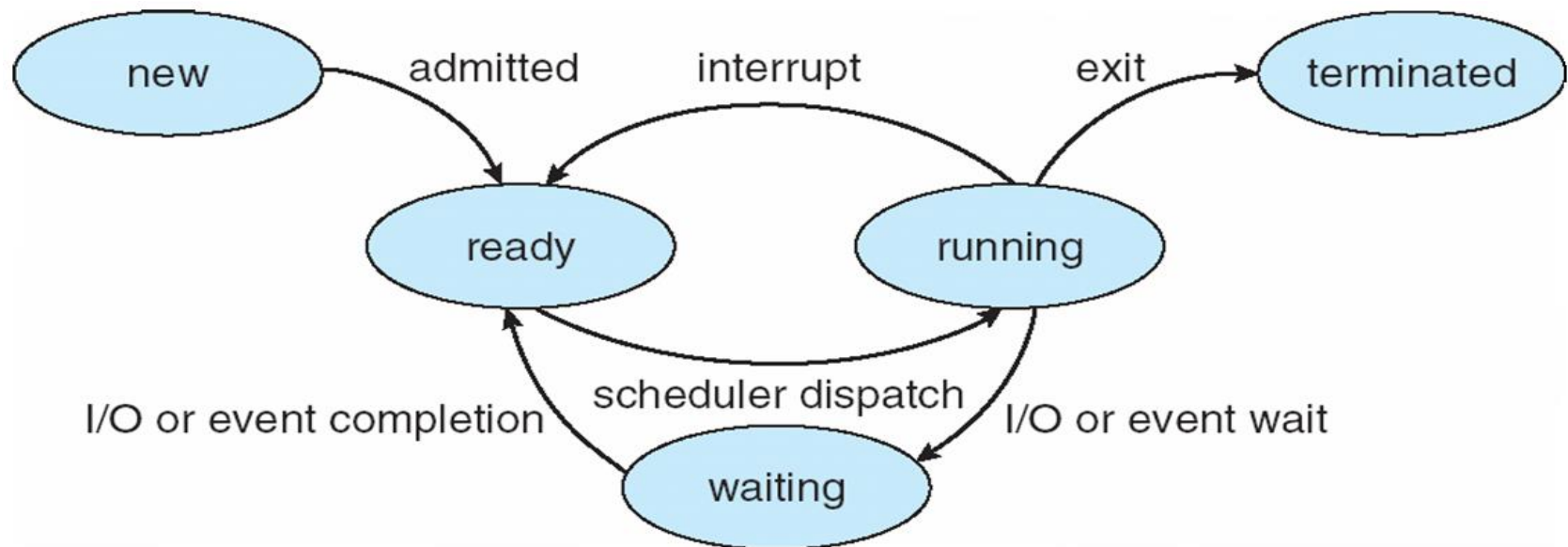
PROCESUL

- Procesul poate fi **caracterizat** de
 - diferitele părți ale sistemului pe care le folosește sau le afectează în decursul execuției
 - Ce poate **citi** și **actualiza** procesul ?
 - **Memorie** – spațiul de adrese este parte a procesului
 - Toate instrucțiunile rezidă în memorie
 - Toate datele citite sau actualizate sunt în memorie
 - **Registrii** – specifici procesorului (PC, SP, etc.) și cei vizibili utilizatorilor -> fac parte din proces
 - **Informațiile de I/E** – ex. fișiere deschise

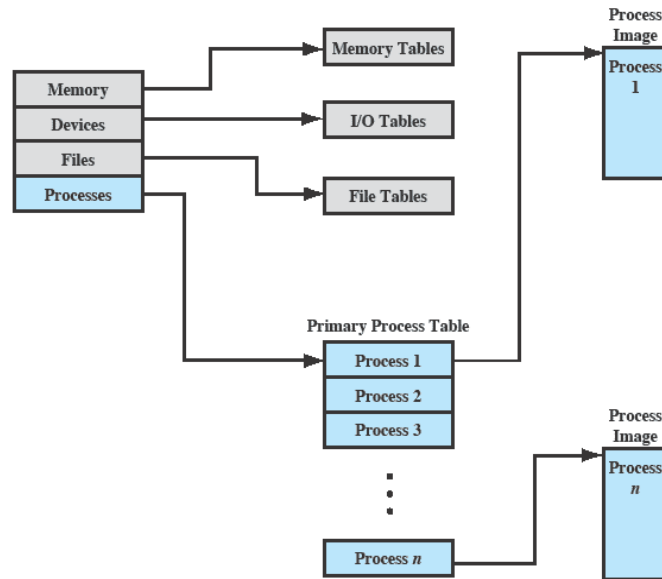
STĂRILE PROCESULUI

- Model cu cinci stări

Silberschatz pg. 103



TABELE DE CONTROL



Stallings, pg. 127

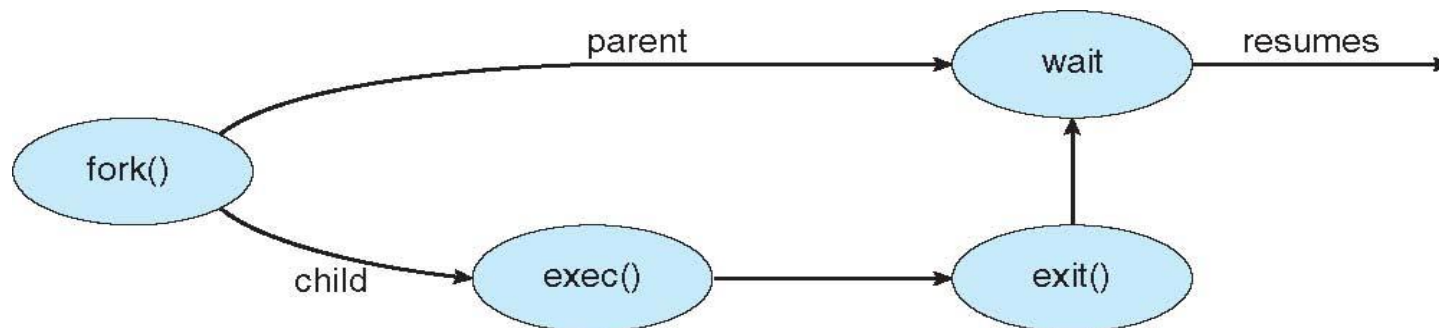
Figure 3.11 General Structure of Operating System Control Tables

- SO este un program -> are nevoie de structuri de date pt. a gestiona informațiile legate de resurse
 - Tabela proceselor - intrare pentru fiecare proces – BCP
 - Identificatorul unic de proces
 - Informații pentru controlul procesului
 - Informații de stare a procesorului

APELURI SISTEM - LINUX

- Identificarea procesului
- Crearea unui proces
- Familia de apeluri **exec**
- Terminarea procesului
- Așteptarea unui proces fiu

Silberschatz, pg. 113



THREADURI

PROCESE - CARACTERISTICI

○ Procesele au două caracteristici majore

• Gruparea resurselor

- Spațiu de adrese virtual care conține imaginea procesului
 - Măsură de protecție – pt evitarea interferenței nedorite dintre procese asupra resurselor
- Grupează resursele pe care le utilizează și/sau le actualizează
 - Segmentele de cod, date și altele (fișiere deschise, semnale și rutine de tratare, etc.)

• Execuție

- Procesul urmează o cale de execuție – posibil intercalată cu alte procese
 - Fiecare proces este în una din stările posibile, are o prioritate asociată și este planificată de SO pentru execuție pe procesor
- Caracterizat de PC – contor de program (indică instrucțiunea de efectuat), regiștri (variabilele curente de lucru) și stiva

THREADURI

- Threadul există în cadrul unui proces
- Permit programului să execute **mai multe operații** în mod (aparent) **simultan**
 - Execuție concurentă
 - Kernelul le planifică în mod asincron
 - Întrerupe fiecare thread din timp în timp -> permite altui thread să fie executat
- Unitate de execuție de **granularitate mai fină** decât procesele
 - Analogie
 - Proces – program în execuție
 - Thread – funcție în execuție

PROCESE ȘI THREADURI

- Threaduri (fire de execuție)

- Descriu un fux de execuție secvențial din cadrul procesului
- Threadurile nu sunt vizibile în afara procesului

- Multithreading

- mai multe fire de execuție paralele (concurente) în cadrul unui proces

(programare cu fire de execuție multiple)

- Mai multe PC-uri (program counter)

- Multiprogramarea și timesharing - multitasking

mai multe procese gestionate simultan

Stallings pg. 159

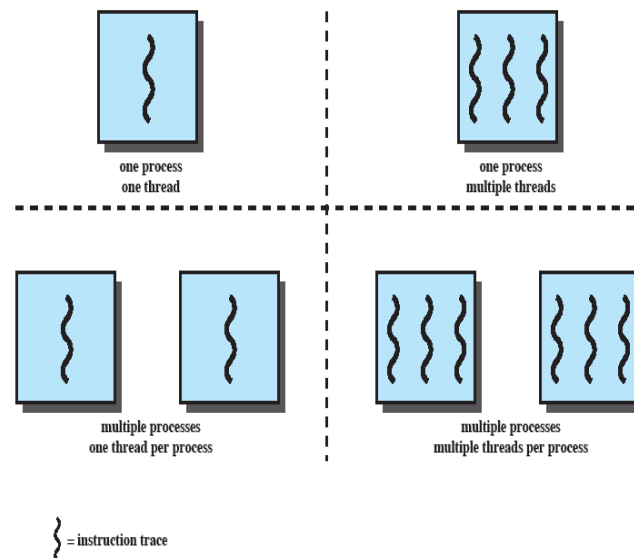


Figure 4.1 Threads and Processes [ANDE97]

PROCESE ȘI THREADURI

○ Crearea unui nou proces – **fork**

- Procesul fiu
 - Execută același program ca și procesul părinte
 - **Spațiul de adrese al procesului părinte, descriptorii de fișier, etc. sunt copiate**
 - Poate să-și modifice segmentul de cod (exec), poate închide descriptori, etc. **fără să afecteze procesul părinte și vice versa**

○ Crearea unui nou thread

- Threadurile **partajează același spațiu de memorie**, descriptori de fișiere și alte resurse ale sistemului
- Dacă un thread
 - Modifică valoarea unei variabile globale, **celelalte threaduri vor vedea această modificare**
 - Închide un descriptor de fișier – celelalte threaduri nu vor putea citi / scrie în fișier
 - Apelează exec – toate **celelalte threaduri sunt terminate**
 - Procesul cu toate threadurile din interior poate executa un singur program deodată

MODELAREA EXECUȚIEI

○ Modelul bazat pe procese

- Două roluri majore
 - Gruparea resurselor
 - Execuție

○ Modelul bazat pe threaduri

- Separă componentele procesului
 - Gruparea resurselor și protecție - rolul principal al **proceselor**
 - Execuție - rolul principal al **threadurilor**
- Threadurile rulează în cadrul proceselor – dar sunt concepte diferite
 - Entitatea de grupare și protecție a resurselor este **procesul** (sau task-ul)
 - Entitatea de planificare și execuție pe procesor este **threadul** (sau proces ușor – lightweight process)

MODELUL THREADURILOR

Resursele alocate procesului

- Spațiu de adrese
- Variabile globale
- Fișiere deschise
- Alarmer în așteptare
- Semnale și rutine de tratare
- Informații de contabilizare

Resursele partajate de toate threadurile procesului

Resurse alocate threadului

- PC – Program Counter
- Regiștri
- Stivă
- Stare

Resurse private fiecărui thread

PROCESE ȘI THREADURI

Stallings pg. 159

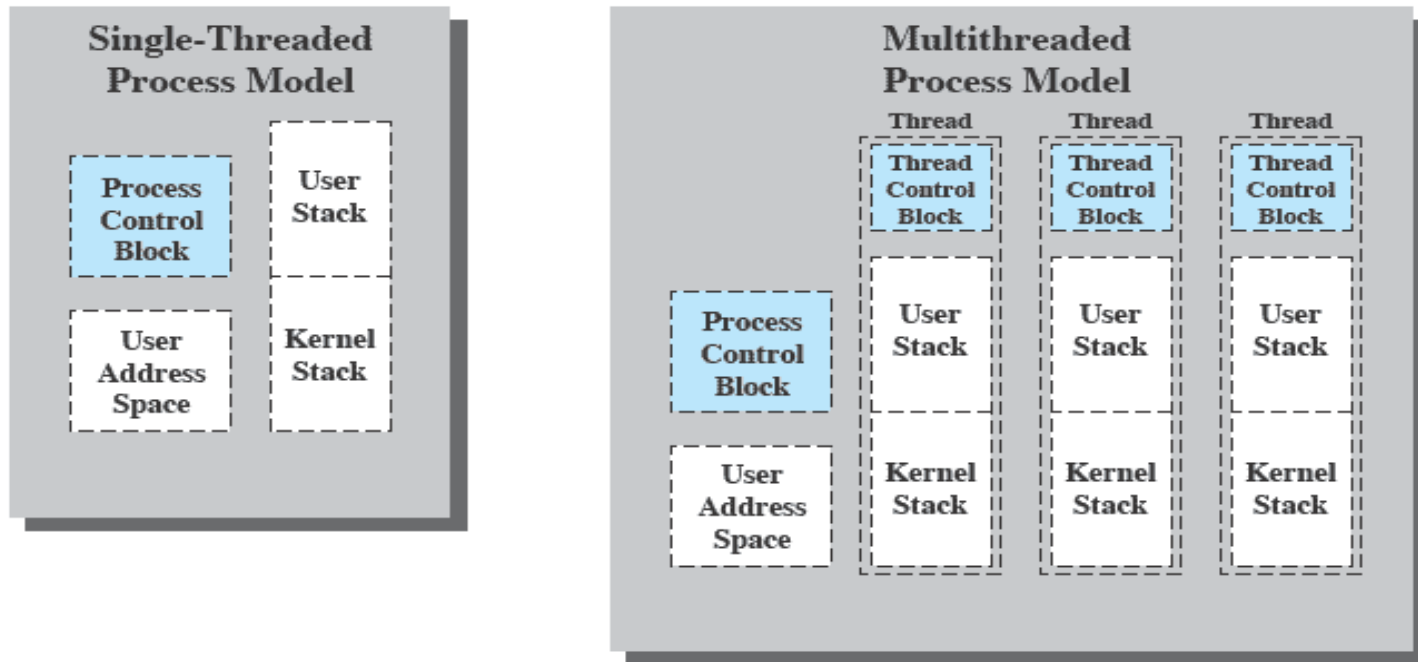
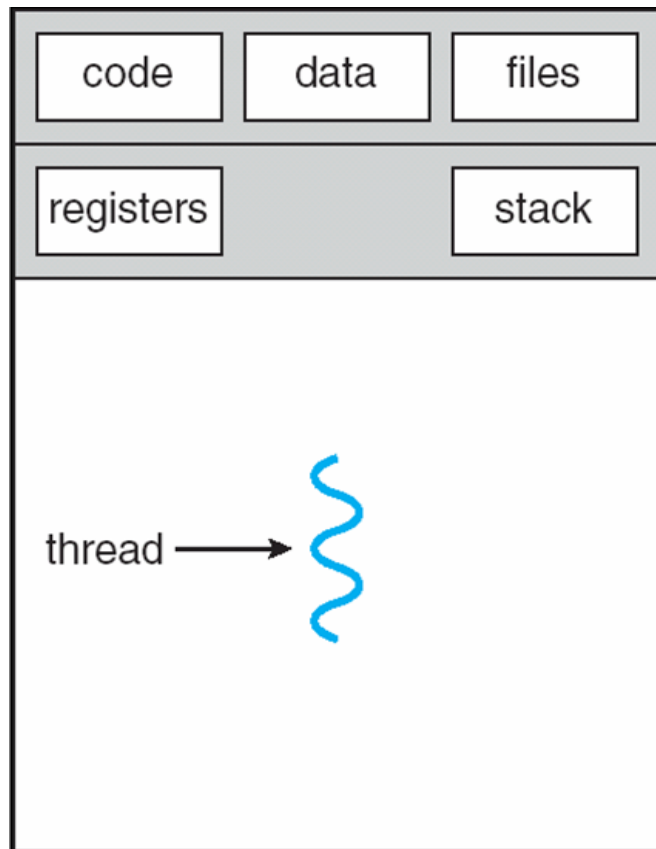


Figure 4.2 Single Threaded and Multithreaded Process Models

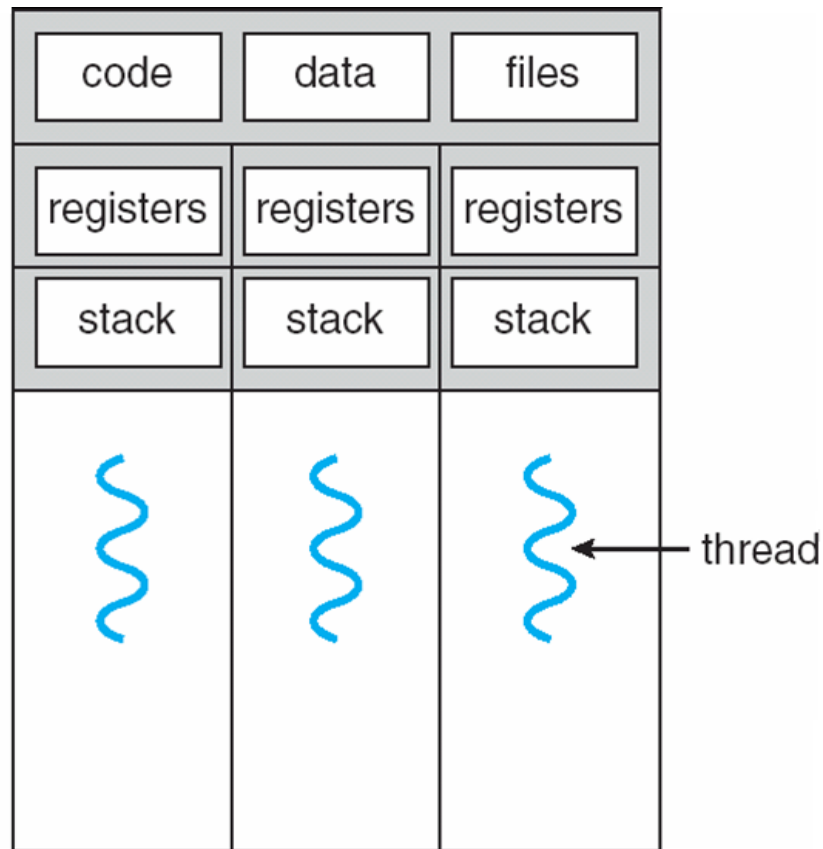
- Proces cu un singur thread
 - Când rulează – controlează regiștrii procesorului
 - Dacă este întrerupt – aceste valori se salvează în PCB
- Fiecare thread are propria stivă și propriul set de regiștri

PROCESE ȘI THREADURI

Silberschatz pg. 154



single-threaded process



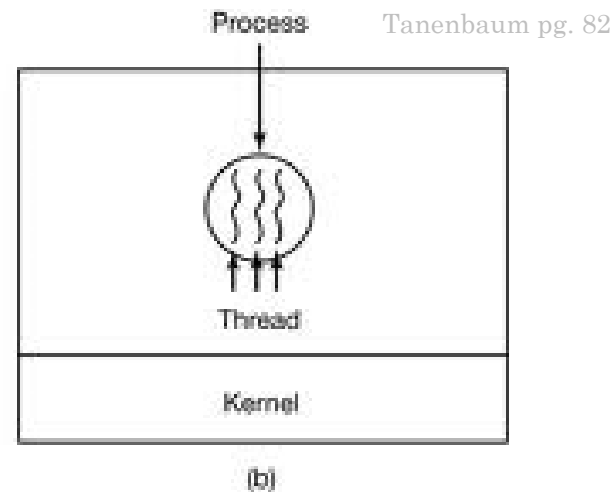
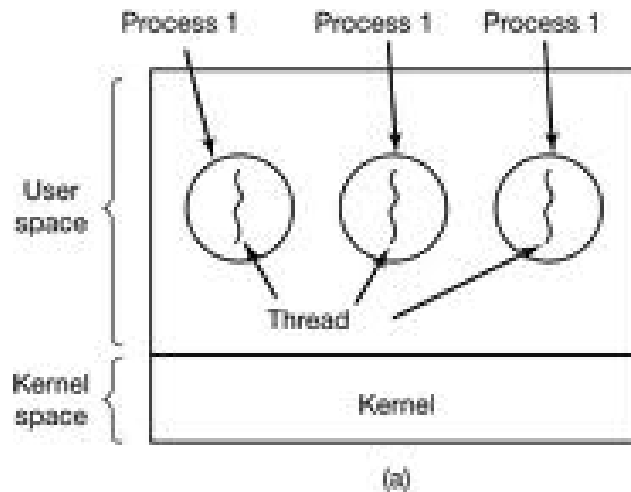
multithreaded process

- Un proces nou are un singur thread
 - Poate crea noi fire de execuție

- Fiecare thread are propria stivă și propriul set de regiștrii

MODELUL THREADURILOR

- **Iluzia paralelismului** pe sisteme monoprosesor
 - Rulează pe rând – comutare rapidă între threaduri
- Toate threadurile din cadrul unui proces **partajează resursele** acestuia
- Threadurile din cadrul unui proces **sunt egale**, dar threadul principal este "mai egal" decât celelalte
- Ex. proces cu trei fire de execuție independente
 - Par să ruleze pe trei procesoare paralele având viteza de trei ori mai mică decât procesorul real



MODELUL THREADURILOR

- Threadurile din cadrul proceselor **nu sunt atât de independente ca și** diferitele **proces**e din cadrul sistemului
 - Partajează același spațiu de adrese
 - Au acces la aceleași variabile globale, fișiere, procese fii, etc.
 - Un thread poate citi, modifica sau șterge întreaga stivă a altui thread
 - Nu există protecție!
 - Motive
 - ESTE IMPOSIBIL
 - NU AR TREBUI SĂ FIE NECESAR
 - Threadurile sunt introduse ca să colaboreze și nu să se atace

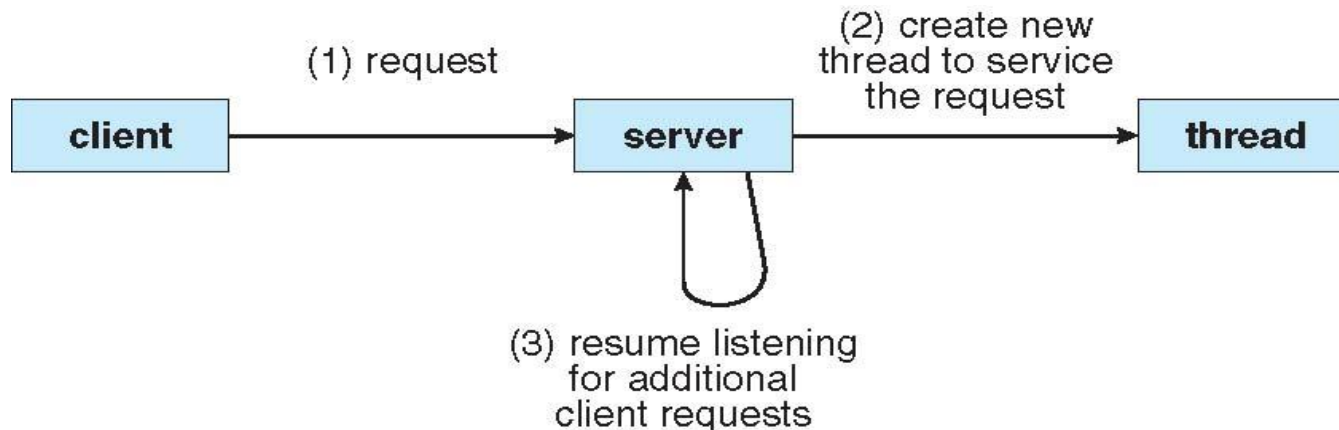
AVANTAJELE THREADURILOR

- Reacție promptă
 - Timp de creare mai mic decât la procese
 - Timp de terminare mai mic decât la procese
 - Comutarea între threaduri durează mai puțin decât comutarea între procese
- Eficiență sporită în comunicarea între threaduri (fără a invoca kernelul)
 - Threadurile aceluiși proces
- Partajarea resurselor – economie
- Scalabilitate
 - Concurență și paralelism nu doar între aplicații ci și în cadrul aplicației
 - poate beneficia de arhitecturile multicore sau multiprocesor

UTILIZAREA THREADURILOR

- Exemple
 - Browser de web
 - Un thread afișează imagini, altul comunică prin rețea
 - Editor de text
 - Un thread afișează grafice, altul răspunde la inputul utilizatorului, al treilea verifică ortografia / gramatica
 - Server de web
 - Acceptă cereri de la mai mulți clienți
 - CUM S-AR IMPLEMENTA DOAR CU PROCESE?
 - Câte un nou thread deservește cererile

Silberschatz pg. 155



MULTITHREADING PE SISTEME MONOPROCESOR

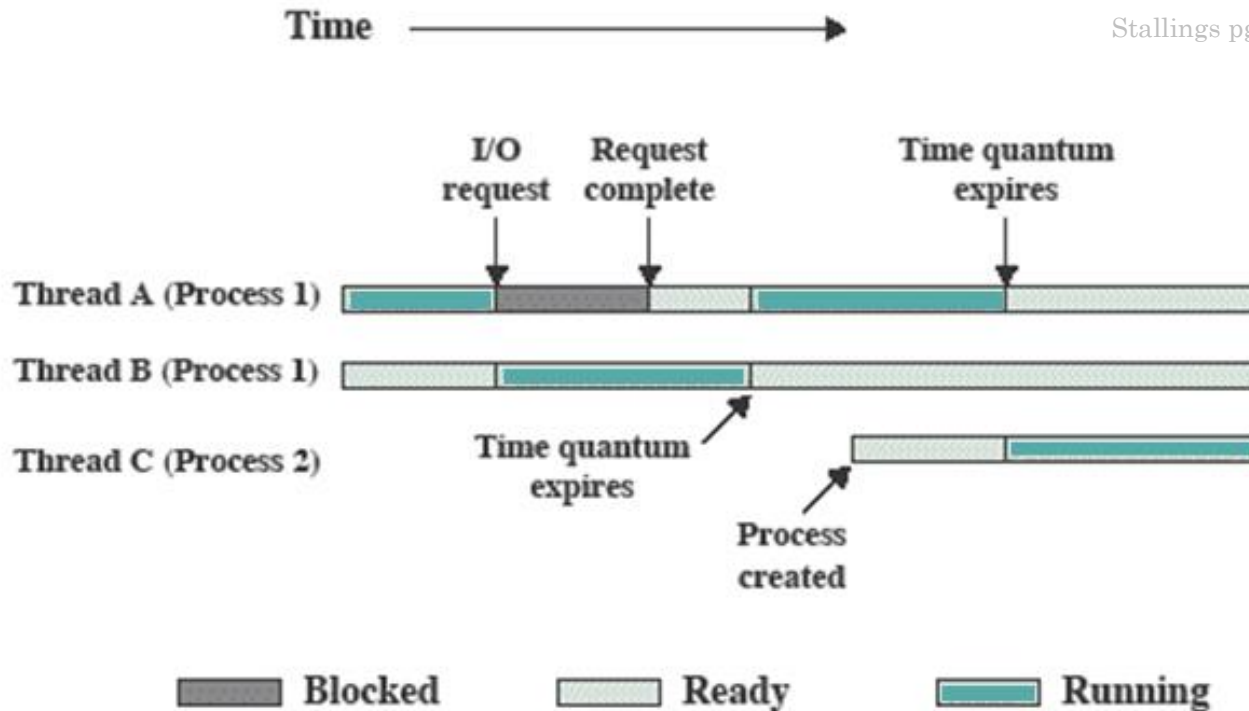


Figure 4.4 Multithreading Example on a Uniprocessor

THREADURI

- Dacă SO suportă gestiunea threadurilor atunci
 - Planificarea și execuția se face la nivel de threaduri
 - Are nevoie de structuri de date care să controleze threadurile
 - Bloc de control al threadurilor
 - Suspendarea temporară a procesului (prin swapping) implică suspendarea tuturor threadurilor din cadrul acelui proces
 - Terminarea procesului implică terminarea fiecărui thread din cadrul acelui proces
- Ce se întâmplă dacă un thread se blochează
 - Blochează sau nu toate celelalte threaduri din proces?

STĂRILE THREADURILOR

○ Stările fundamentale ale threadurilor

- În execuție (**Running**)
 - activ
- Gata de execuție (**Ready**)
 - planificat dar inactiv
- Blocat (**Blocked**)
 - În așteptarea unui eveniment extern

○ Analogic cu stările proceselor

○ Operații asociate cu schimbarea de stare a unui thread

- Creare
- Blocare
- Deblocare
- Terminare

IMPLEMENTAREA THREADURILOR

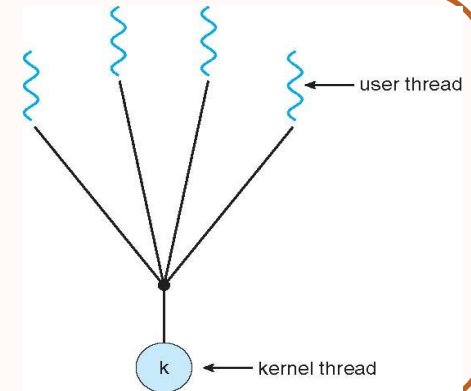
- În spațiul utilizator – user-level threads
- În kernel – kernel level threads (lightweight processes)
- Combinat (metodă hibridă)

MULTITHREADING - MODELE

Silberschatz pg. 157

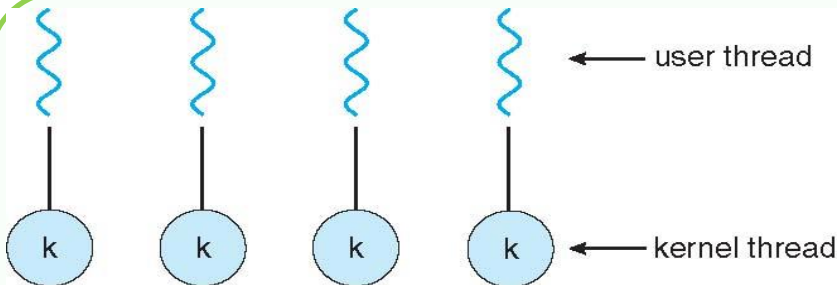
Many-to-one

- Mai multe fire de nivel utilizator mapate la un fir kernel



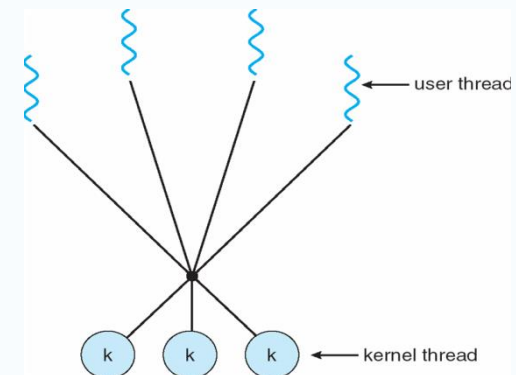
One-to-one

- Unu la unu – fiecărui thread de nivel utilizator îi corespunde un thread din kernel



Many-to-many

- Mai multe threaduri utilizator sunt mapate la un număr (mai mic) de threaduri kernel

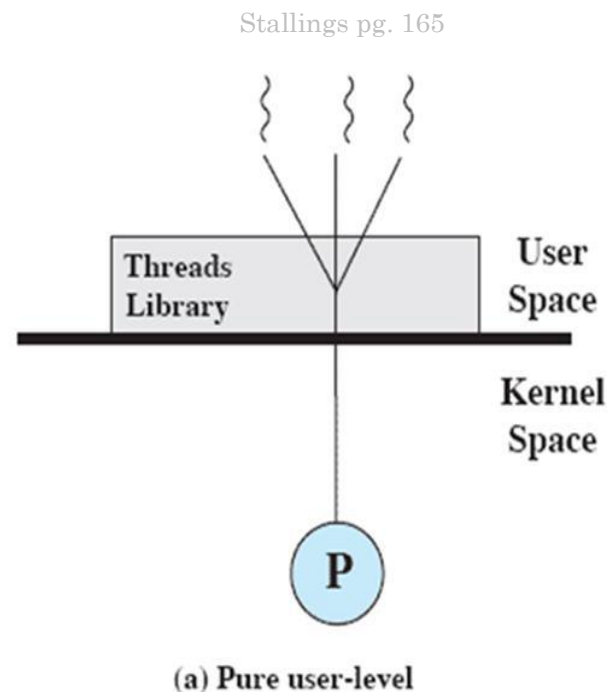


THREADURI ÎN SPAȚIUL UTILIZATOR

- Gestiunea se realizează în spațiul utilizator – de către aplicație
 - Kernelul nu știe despre existența threadurilor
- Fiecare proces are propria **tabelă de threaduri**
 - Reține informații legate de threadurile din proces

○ Avantaje

- Comutarea între threaduri nu necesită invocarea kernelului
 - Apel de procedură, fără întreruperi, fără schimbare de context – **foarte rapid**
- Planificare specifică aplicației
 - Fiecare proces poate avea un algoritm de planificare propriu
- Potrivit și pentru SO care nu oferă suport pentru threaduri

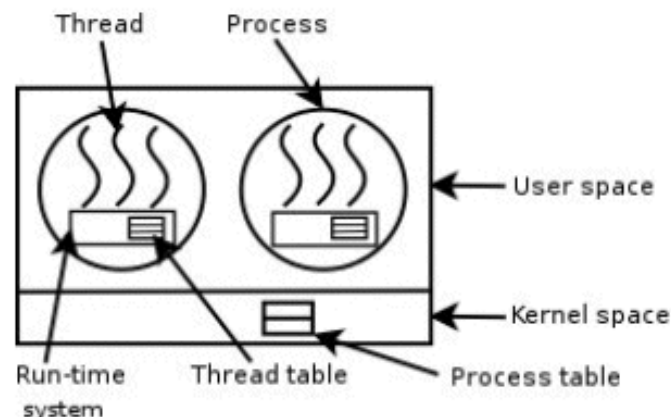


THREADURI ÎN SPAȚIUL UTILIZATOR

- Comutarea între threaduri analogic cu o schimbare de context

- Dar mai eficient
 - Cu cel puțin o ordine de mărime mai rapid decât comutarea între procese

- Context: regiștrii utilizator, PC, SP



Tanenbaum pg. 91

- Dezavantaje

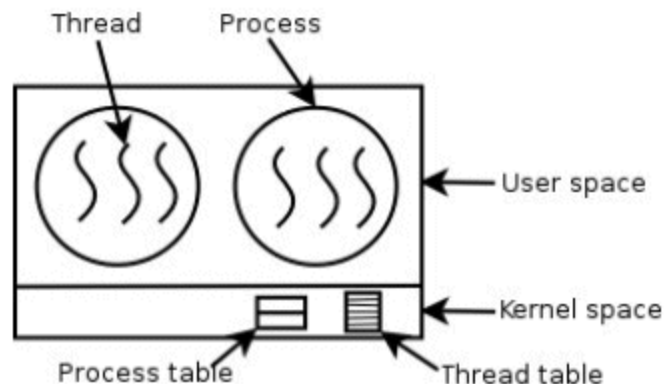
- Implementarea **apelurilor sistem blocante**
 - Blocarea unui thread blochează toate celelalte threaduri din proces
 - Compromite scopul introducerii threadurilor
 - Aplicația, deși cu mai multe fire de execuție - **Nu poate profita de arhitectura cu mai multe procesoare** (multi-processor, multi-core)
 - Threadul trebuie să **predea ștafeta** în mod voluntar

- Soluții

- Rescrierea apelurilor – pentru a nu fi blocante
 - Necesită schimbări la nivelul SO – neadecvat
 - Testarea blocării - jacketing

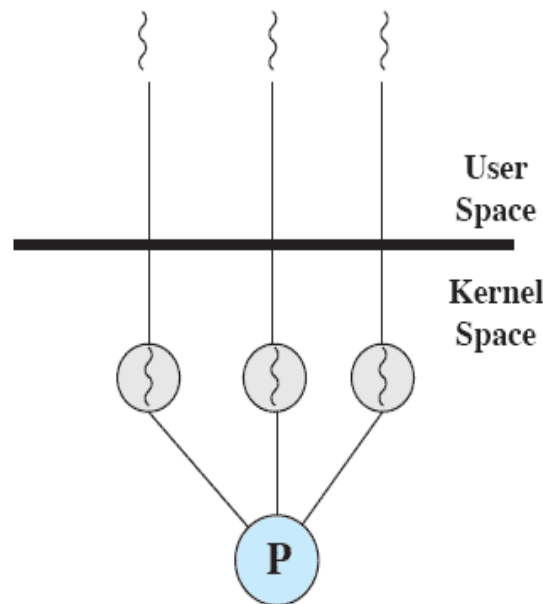
THREADURI ÎN **KERNEL**

- Gestiunea se realizează de către kernel
 - API pentru accesul la threadurile kernel
- Kernelul gestionează tabela proceselor și tabela threadurilor
 - Pt. toate threadurile din sistem
- Planificare la nivel de threaduri
- **Avantaje**
 - Kernelul poate planifica mai multe threaduri din cadrul aceluiași proces simultan (pe mai multe procesoare)
 - Dacă un thread este blocat kernelul poate planifica alt thread din același proces (sau alt proces)
 - Nu apare problema apelurilor blocante
 - Rutinele kernelului pot folosi mai multe fire de execuție



Tanenbaum pg. 91

Stallings pg. 165



(b) Pure kernel-level

THREADURI ÎN KERNEL

○ Dezavantaje

- Comutarea între threaduri necesită intervenția kernelului
- Crearea și distrugerea threadurilor sunt mai costisitoare în timp și resurse (față de threadurile utilizator)
 - Soluție: reciclarea threadurilor (thread pool)

Timpul (μsec) necesar pt crearea, planificare, executarea și terminarea unui thread / proces care invocă o procedura vidă

Stallings pg. 168

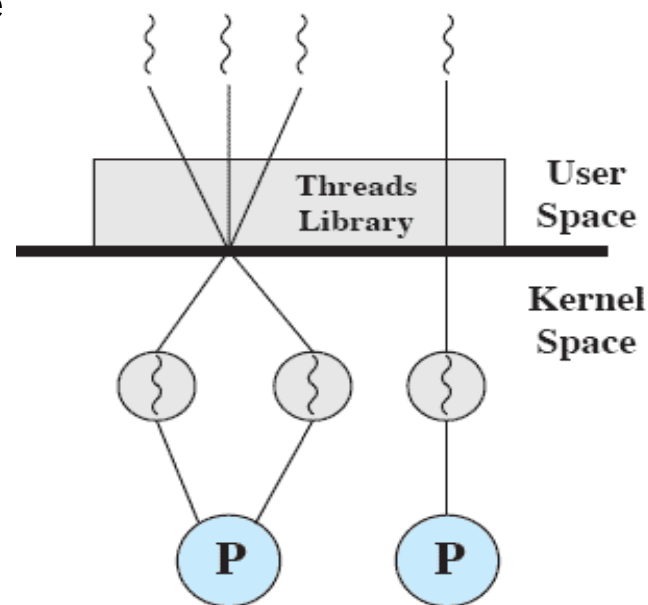
Operation	User-Level Threads	Kernel-Level Threads	Processes
Null Fork	34	948	11,300
Signal Wait	37	441	1,840

Timpul necesar pt sincronizare – (signal + wait)

METODA HIBRIDĂ

Stallings pg. 165

- Combină avantajele metodelor anterioare
- Mai multe threaduri din spațiul utilizator sunt **mapate** pe un număr (mai mic sau egal) de threaduri din kernel
- Kernelul e conștient doar de threadurile kernel – pe acestea le planifică
 - Crearea, planificarea și sincronizarea threadurilor mapate pe fire kernel se realizează în spațiul utilizator
- Mai multe threaduri pot fi **rulete** **simultan** pe mai multe procesoare
- Apelurile blocante **nu blochează** celelalte threaduri



(c) Combined

REZUMAT

- Dacă procesul este un **program în execuție**, atunci threadul este o **funcție în execuție**
 - Într-un mediu multithreaded, procesul este unitatea de alocare a resurselor și de protecție, iar threadul este unitatea de execuție
- Comunicare
 - Threadurile din același proces pot comunica prin spațiul de adrese partajat (partajează resursele procesului)
- Eficiență și scalabilitate
 - O aplicație cu multiple fire de execuție realizează concurență fără încărcarea specifică utilizării mai multor procese
 - Concurență și paralelism nu doar între aplicații ci și în cadrul aplicației
 - poate beneficia de arhitecturile multicore sau multiprocesor
- Implementare
 - La nivelul utilizator
 - La nivelul kernelului
 - Combinat

EFFECTUL APELURILOR FORK ȘI EXEC

- Semantica apelurilor **fork** și **exec** se schimbă în cadrul programelor cu mai multe fire de execuție
- Dacă un thread al procesului apelează **fork**
 - Noul proces va avea câte o copie a fiecărui thread din procesul părinte? sau
 - Noul proces va avea un singur fir de execuție?
- Unele sisteme UNIX (Solaris) au **două versiuni de fork**
 - Unul care dublează fiecare thread
 - Altul care dublează doar threadul din care s-a apelat **fork**
 - Alegerea se face în funcție de aplicație
- Dacă se apelează **exec** dintr-un thread al procesului, noul program va înlocui întregul proces
 - incluzând toate threadurile

THREADURI LINUX

- Linux nu face diferențiere între procese și threaduri – folosește termenul **task**
 - Threadurile de nivel utilizator sunt mapate în procese din kernel – cu același GID
 - Permite partajarea resurselor și evită încărcarea aferentă schimbării de context între aceste threaduri (procese din același grup)
- Apelul sistem **clone**
 - Noul proces partajează resursele procesului părinte
 - Opțiunile controlează gradul de partajare
 - Cele două procese se comporta ca și threadurile aceluiași proces
 - Nu sunt definite noi tipuri de date pentru threaduri

Silberschatz pg. 173

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

BIBLIOTECI PENTRU GESTIUNEA THREADURILOR

- Furnizează API pentru gestiunea threadurilor
- 2 modalități
 - Bibliotecă integrată complet în spațiul utilizator (nivel aplicație)
 - Bibliotecă integrată în kernel cu suport din partea SO
- Ex.
 - **Pthreads** (POSIX threads) – în ambele modalități
 - **Win32 API** – nivel kernel
 - **Java Threads** – folosește metoda utilizată de sistemul gazdă

GESTIUNEA THREADURILOR CU PTHREADS

○ Exemplu de creare a două threaduri

```
#include <stdio.h>
#include <assert.h>
#include <pthread.h>
```

```
void *mythread(void *arg) {
    printf("%s\n", (char *) arg);
    return NULL;
}
```

```
int
main(int argc, char *argv[]) {
    pthread_t p1, p2;
    int rc;
    printf("main: begin\n");
    rc = pthread_create(&p1, NULL, mythread, "A"); assert(rc == 0);
    rc = pthread_create(&p2, NULL, mythread, "B"); assert(rc == 0);
    // join waits for the threads to finish
    rc = pthread_join(p1, NULL); assert(rc == 0);
    rc = pthread_join(p2, NULL); assert(rc == 0);
    printf("main: end\n");
    return 0;
}
```

EXEMPLU DE CREARE A DOUĂ THREADURI

- Rezultat
posibil nr. 1

```
main starts running  
main prints "main: begin"  
main creates thread T1  
main creates thread T2  
main waits for T1
```

```
T1 runs  
T1 prints "A"  
T1 returns
```

```
main waits for T2
```

```
T2 runs  
T2 prints "B"  
T2 returns
```

```
main prints "main: end"
```

EXEMPLU DE CREARE A DOUĂ THREADURI

○ Rezultat posibil nr. 1

```
main starts running
main prints "main: begin"
main creates thread T1
main creates thread T2
main waits for T1
```

```
T1 runs
T1 prints "A"
T1 returns
```

```
main waits for T2
```

```
T2 runs
T2 prints "B"
T2 returns
```

```
main prints "main: end"
```

```
main starts running
main prints "main: begin"
main creates thread T1
```

```
T1 runs
T1 prints "A"
T1 returns
```

```
main creates thread T2
```

```
T2 runs
T2 prints "B"
T2 returns
```

```
main waits for T1
  (returns immediately because T1 is finished)
main waits for T2
  (returns immediately because T2 is finished)
main prints "main: end"
```

○ Rezultat posibil nr. 2

EXEMPLU DE CREARE A DOUĂ THREADURI

- Rezultat
posibil nr. 3

```
main starts running
main prints "main: begin"
main creates thread T1
main creates thread T2

T2 runs
T2 prints "B"
T2 returns

main waits for T1

T1 runs
T1 prints "A"
T1 returns

main waits for T2
  (returns immediately because T2 is finished)
main prints "main: end"
```

- Nu știm ordinea în care se vor executa threadurile

EXEMPLU DE CREARE A DOUĂ THREADURI

- Rezultat posibil nr. 3

```
main starts running
main prints "main: begin"
main creates thread T1
main creates thread T2
```

```
T2 runs
T2 prints "B"
T2 returns
```

```
main waits for T1
```

```
T1 runs
T1 prints "A"
T1 returns
```

```
main waits for T2
  (returns immediately because T2 is finished)
main prints "main: end"
```

- Nu știm ordinea în care se vor executa threadurile
 - Și aceasta nu este tot...
 - lucrurile se complică și mai tare....
 - Accesul la resursele partajate

EXEMPLU DE INCREMENTARE CONCURENTĂ

```
#include <stdio.h>
#include <pthread.h>
#include "mythreads.h"

static int counter = 0;

//
// mythread()
//
// Simply adds 1 to counter repeatedly, in a loop
// No, this is not how you would add 10,000,000 to
// a counter, but it shows the problem nicely.
//
void *
mythread(void *arg)
{
    printf("%s: begin\n", (char *) arg);
    int i;
    for (i = 0; i < 1e7; i++) {
        counter = counter + 1;
    }
    printf("%s: done\n", (char *) arg);
    return NULL;
}
```

```
//
// main()
//
// Just launches two threads (pthread_create)
// and then waits for them (pthread_join)
//
int
main(int argc, char *argv[])
{
    pthread_t p1, p2;
    printf("main: begin (counter = %d)\n", counter);
    pthread_create(&p1, NULL, mythread, "A");
    pthread_create(&p2, NULL, mythread, "B");

    // join waits for the threads to finish
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    printf("main: done with both (counter = %d)\n", counter);
    return 0;
}
```

EXEMPLU DE CREARE A DOUĂ THREADURI

○ Rez. posibil nr. 1

- Rezultatul
așteptat

```
prompt> gcc -o main main.c -Wall -pthread
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 20000000)
prompt>
```

EXEMPLU DE CREARE A DOUĂ THREADURI

○ Rez. posibil nr. 1

- Rezultatul așteptat

```
prompt> gcc -o main main.c -Wall -pthread
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 20000000)
prompt>
```

```
prompt> gcc -o main main.c -Wall -pthread
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19345221)
```

○ Rez. posibil nr. 2

- Rezultat eronat

EXEMPLU DE CREARE A DOUĂ THREADURI

○ Rez. posibil nr. 1

- Rezultatul așteptat

```
prompt> gcc -o main main.c -Wall -pthread
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 20000000)
prompt>
```

```
prompt> gcc -o main main.c -Wall -pthread
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19345221)
```

○ Rez. posibil nr. 2

- Rezultat eronat

○ Rez. posibil nr. 3

- Rezultat eronat
- diferit

```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19221041)
prompt>
```

○ Problema: planificare necontrolată

EXEMPLU: PANIFICARE NECONTROLATĂ

- Instrucțiunile nu sunt atomice
 - Întreruperea poate apare oricând

OS	THREAD 1	THREAD 2	(values AFTER inst)		
			PC	%eax	COUNTER
	(before crit sect)		100	0	50
	mov 0x8049a1c, %eax		105	50	50
	add \$0x1, %eax		108	51	50
Interrupt	(save T1's state,				
restore T2)			100	0	50
		mov 0x8049a1c, %eax	105	50	50
		add \$0x1, %eax	108	51	50
		mov %eax, 0x8049a1c	113	51	51
Interrupt	(save T2's state,				
restore T1)			108	51	51
	mov %eax, 0x8049a1c		113	51	51
					(not 52!)

Figure 25.5: The Problem: Up Close and Personal